Royale Cars — Plano de unificação das três plataformas

Versão 2 | Oct 13, 2026 · Grupo Royale Cars: Caio (desktop), Gabriel (web), Evaldo (mobile)

Visão geral

Hoje o Royale Cars é um sistema contado de três jeitos. A web (PHP) tem as 32 funcionalidades e as regras mais completas, mas grava num MySQL que só existe no computador de quem roda. O desktop (Electron) e o celular (Flutter) já usam o mesmo projeto Supabase, mas gravam com máscara, têm login próprio e faltam 19 e 20 funcionalidades, respectivamente.

A proposta deste documento: **um banco só no Supabase, onde ficam as regras** (validação, permissões, cálculo do aluguel e histórico), e **três aplicativos que cuidam só das telas**. A web serve de modelo para as regras; o desktop e o celular entram com o que só eles têm (grupos de permissão, quilometragem, assinatura, contrato em PDF e checklist).

> **O que mudou nesta versão 2**
> As 13 decisões foram respondidas, a entrega foi marcada para **27/10** e o repositório já existe. Por isso o plano foi comprimido de 5 a 7 semanas para 14 dias: as fases agora têm datas, as plataformas são conectadas em paralelo depois que o banco sobe, e a fase de completar foi dividida em prioridades (A, B e C).

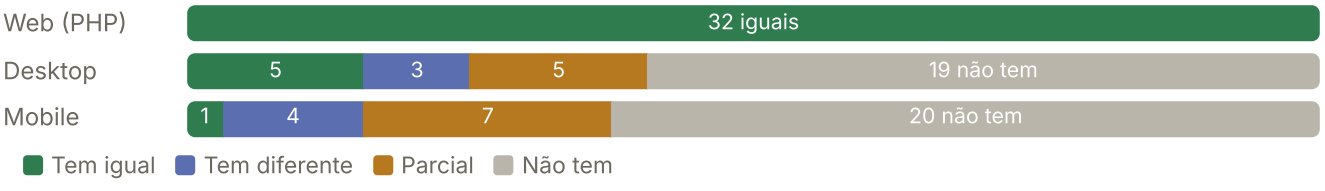
O trabalho está dividido em **7 fases e cerca de 45 pacotes**. Cada pacote é uma mudança pequena, testada e com mensagem de commit pronta, para o histórico do GitHub mostrar o projeto evoluindo passo a passo.

Como usar este documento:

1. Leiam **Onde cada plataforma está** e **O que eu conferi nos códigos**: é o diagnóstico.
2. As seções **Arquitetura**, **O que levar de cada plataforma** e **Padrão comum** são o que foi decidido.
3. **Calendário até 27/10** e **Pacotes** são o planejamento: o que vem antes, quem faz o commit e quando o banco precisa de vocês.
4. **Como fazer os commits sem saber Git** responde a parte do GitHub.
5. A última seção, **O que eu preciso de vocês agora**, é o que destrava o início.

Onde cada plataforma está

Contagem do checklist de 32 funcionalidades da web, tirada dos três documentos de vocês:



Assunto	Web (PHP)	Desktop (Electron)	Mobile (Flutter)
Banco	MySQL local (XAMPP), 8 tabelas	Supabase, 5 tabelas	Mesmo Supabase do desktop + 1 tabela e colunas próprias
Login	Tabela própria, bcrypt no PHP, sessão no servidor	Tabela `usuarios`, bcrypt dentro do app	Igual ao desktop; não confere conta desativada
Permissões	3 cargos, 13 chaves por pessoa	2 cargos, 14 chaves por grupo	Nenhuma: todo mundo vê tudo
Formato dos dados	Sem máscara (só números)	Com máscara	Com máscara
Status	`Em manutenção`, `Em andamento` / `Finalizada` / `Cancelada`	`Em Manutenção`, `Ativo` / `Concluído`	Igual ao desktop
Onde ficam as regras	PHP + MySQL	No app, poucas	No app, médias
Só ela tem	Manutenção, alertas, relatórios, histórico, painel do chefe	Grupos de permissão, bandeja do sistema, multiempresa	Quilometragem, assinatura, contrato PDF, checklist, taxas

Em resumo: a web está mais completa, mas é a que está mais longe do banco. O desktop e o celular já estão no banco, mas estão longe das regras.

O que eu conferi nos códigos

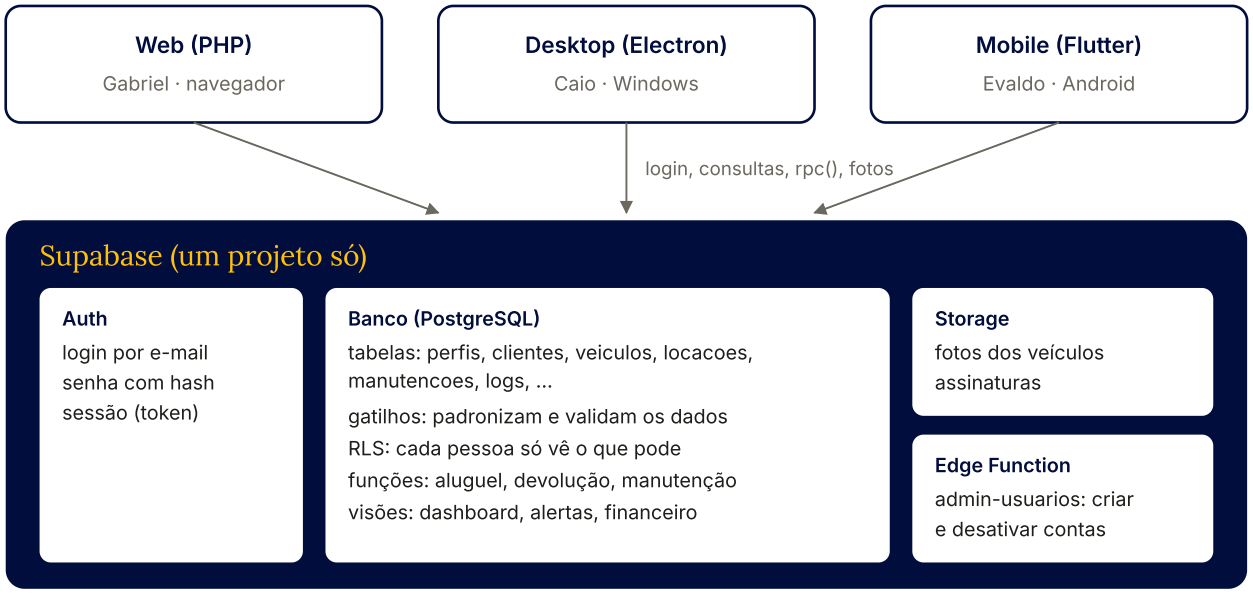
Abri os três zips para confirmar os PDFs. Eles batem com o código. Encontrei seis pontos que mudam o plano:

O que encontrei	Por que importa	O que fazemos
O script do banco unificado não veio no zip. O documento da web cita `supabase/royale_cars_supabase.sql`, mas o zip só tem o `royale_cars.sql` do MySQL.	Toda a unificação depende desse script.	Vocês confirmaram que têm. É a base da Fase 1.
O zip da web tem duas versões. `royale-cars-web` (1º de setembro, antiga) e `Royale-Cars-TCC/royale-cars-web` (9 de outubro, a completa).	Se alguém copiar a pasta errada, perde metade do sistema.	Usar só a de 9 de outubro. A antiga não entra no repositório.
O zip do mobile tem 169 MB, e 157 MB são a pasta `build/`.	Essa pasta é gerada sozinha pelo Flutter. Se for para o GitHub, o repositório fica pesado e o push pode falhar.	Colocar `build/` no `.gitignore` (pacote 0.1).

O que encontrei	Por que importa	O que fazemos
O repositório `projeto_royale_cars` só tem zips (desktop v3.2 e v3.3, web, mobile v13), enviados pelo site. O zip da web de lá é a versão antiga, de 1º de setembro.	Os professores não conseguem ler o código dentro de um zip, e o Git não mostra o que mudou de um commit para o outro.	A Fase 0 troca os zips pelas pastas de código, com a versão nova da web.
A chave do Supabase está escrita no código do desktop e do celular, e o celular lê a linha inteira de `usuarios`, inclusive o hash da senha.	Essa chave (`publishable`) pode ser pública, *desde que o RLS esteja ligado*. Se estiver desligado, qualquer pessoa com o app lê e altera tudo.	A Fase 1 liga o RLS em todas as tabelas. Até lá, conferir no painel (consulta na lista do final).
O que eu consigo testar aqui: tenho PHP 8.3, PostgreSQL 16 e Node 22. Não tenho Flutter.	Consigo rodar o script do banco num Postgres de teste e conferir PHP e JavaScript antes de entregar. O Dart eu não consigo compilar.	Nos pacotes do mobile, o Evaldo roda `flutter run` e me manda os erros, se houver.

Arquitetura: regras no banco, telas nos apps

Pensem no banco como a **portaria de um prédio**. Hoje cada app tem o seu porteiro: o da web confere CPF, o do desktop não confere, o do celular confere pela metade. Quem entra pelo porteiro mais fraco grava o dado errado, e os outros dois veem o erro. A solução é ter **um porteiro só, dentro do banco**. Os apps continuam com as suas telas e máscaras (para ajudar quem digita), mas quem decide se o dado entra é o banco.



Os três apps falam com o mesmo Supabase. Tudo que é regra fica na caixa azul, escrito uma vez só, em SQL.

Por que trocar o login próprio pelo Supabase Auth

É a maior mudança para o desktop e o celular, então vale explicar. Com o login próprio, o app usa a mesma chave pública para todo mundo; o banco não sabe *quem* está pedindo os dados, então não

consegue aplicar permissão nenhuma. Com o Supabase Auth, cada pessoa recebe um "crachá" (token) ao entrar, e o banco passa a saber quem é quem.

Ponto	Manter login próprio (tabela `usuarios`)	Supabase Auth (recomendado)
Quem protege os dados	Só a tela do app. Pela chave pública dá para ler tudo, inclusive os hashes de senha.	O banco (RLS). Desativar alguém corta o acesso na hora, nas três plataformas.
Trabalho em cada app	Nenhum agora, mas cada app continua conferindo senha, bloqueio e permissão sozinho (três vezes).	Uma chamada (`signInWithPassword`) nos SDKs de JavaScript e Dart; na web, uma requisição HTTP.
Senhas atuais	Continuam.	O Supabase aceita importar hash bcrypt, então ninguém precisa criar senha nova (se os dados forem mantidos).
Na banca	Difícil defender a segurança.	Dá para mostrar: "desativei o Lucas na web e o celular dele saiu".

O que muda para cada app, em uma frase

- **Desktop:** continua com o `supabase-js`; troca o login, passa a chamar as funções do banco (`supabase.rpc`) para aluguel e manutenção, e lê as visões no dashboard.
- **Web:** sai do MySQL. O PHP passa a falar com o Supabase pela API (um arquivo `includes/supabase.php` com funções como `sb_select` e `sb_rpc`). As telas ficam quase iguais; o que muda é a parte que busca e grava os dados.
- **Mobile:** continua com o `supabase_flutter`; troca o login, tira o `id_empresa` das consultas e passa a usar as mesmas funções e visões.

O que levar de cada plataforma

Nenhuma das três está "certa" sozinha. A web é a base das regras, mas o desktop e o celular têm ideias que a banca vai gostar de ver nas três.

Da web

a base de tudo

- Formatos gravados no banco (CPF, telefone, placa sem máscara)
- Validações: CPF e CNH com dígito, DDD, placa, chassi, nome
- Status do veículo só por ações
- Cálculo do aluguel com diária gravada, data real e atraso
- Cancelamento com motivo
- Módulos: Manutenção, Alertas, Financeiro, Relatórios + CSV
- Segurança: bloqueio por tentativas, senha temporária, saída por inatividade
- Painel do chefe: histórico de ações e tentativas de login

Do desktop

organização da equipe

- Grupos de permissão como **modelos** (Gerente, Vendedor, Financeiro) que preenchem as caixas da pessoa
- Tela "Ver colaboradores" do grupo e mover de grupo
- Endereço do cliente (útil no contrato)
- Modal de confirmação padrão (vermelho = perigo, verde = confirmar)
- Botão "Arquivados" que alterna a lista
- Listas que atualizam sozinhas
- Bandeja do sistema (só desktop)

Do mobile

o atendimento no balcão

- **Quilometragem** no veículo e na locação (pedido da banca)
- **Assinatura digital** do cliente (pedido da banca)
- **Contrato em PDF** com a assinatura, que vira o comprovante das três
- **Checklist de inspeção** (motor, freios, pneus, óleo) dentro da manutenção
- Taxas extras na devolução, com motivo
- Relatório de desempenho (taxa de ocupação)
- Sobre Nós (apresenta o grupo)

O que sai das três

Sai	Onde existe hoje	Por quê
"Criar conta" público	Desktop e mobile	Qualquer pessoa criava uma locadora. Só o chefe cadastra pessoas.
"Esqueci minha senha" só com o e-mail	Desktop	Quem soubesse o e-mail de alguém trocava a senha dele. O chefe redefine com senha temporária.
Multiempresa (`id_empresa`)	Desktop e mobile	Se o grupo escolher empresa única (decisão 1), simplifica o banco e todas as consultas.
Status do veículo no formulário	Desktop e mobile	Era o que deixava carro "preso" em manutenção com aluguel aberto.
Login por nome de usuário	Desktop e mobile	O Supabase Auth entra por e-mail.
`bcrypt` dentro dos apps	Desktop e mobile	A senha passa a ser conferida pelo Supabase Auth.
Formulário de Contato que finge enviar	Mobile	Na banca, um botão que não faz nada pega mal.

Padrão comum

As regras detalhadas (algoritmos de CPF e CNH, limites de datas, cálculo) continuam valendo como estão no documento da web. Aqui fica só o resumo do que as três precisam escrever igual.

Assunto	Padrão
Formato no banco	CPF e CNH com 11 números; telefone com DDD, só números; e-mail em minúsculas; placa em maiúsculas sem hífen; chassi com 17 caracteres; dinheiro com 2 casas; datas ano-mês-dia. A máscara é só na tela.
Status do veículo	`Disponível` · `Alugado` · `Em manutenção` (m minúsculo). Muda só por ações.
Status da locação	`Em andamento` · `Finalizada` · `Cancelada`. "Atrasada" é calculado, não gravado.
Status da manutenção	`Em andamento` · `Concluída`
Cargos	`chefe` · `gerente` · `funcionario`
Permissões	As 13 chaves da web, por pessoa. Os grupos do desktop viram modelos que marcam as caixas.
Senha	Mínimo de 8 caracteres, com letra e número.
Cores	Azul `#010d3d` e dourado `#FFC107` (as três já usam).

Nomes de tela (aprovado)

Hoje a mesma tela tem nomes diferentes. Ficou decidido usar o nome que já aparece em duas das três plataformas, ou o que bate com o nome da tabela.

Web hoje	Desktop hoje	Mobile hoje	Nas três
Aluguéis	Locações	Nova Locação / Devoluções	**Locações** (tabela `locacoes`)
Veículos	Frota	Frota / Cadastro Veículos	**Frota**
Financeiro	Financeiro	Finanças	**Financeiro**
Painel Administrativo	Administração	(não tem)	**Administração**
Minha conta	Configurações	(não tem)	**Minha conta**

Banco unificado: as tabelas

É o banco da proposta da web com o que o desktop e o celular trazem. As colunas novas em relação à web estão em negrito.

Tabela	O que guarda	Veio de
`perfis`	Nome, e-mail, cargo, ativo, troca de senha obrigatória, último acesso (o login fica no Supabase Auth)	Web

Tabela	O que guarda	Veio de
`usuario_permissoes`	As 13 permissões de cada pessoa	Web
`modelos_permissao`	Grupos-modelo: nome, descrição e lista de chaves	Desktop (grupos)
`clientes`	Nome, CPF, CNH, validade da CNH, e-mail, telefone, **endereço**, arquivado	Web + desktop
`veiculos`	Marca, modelo, ano, placa, chassi, diária, foto, status, **quilometragem**, arquivado	Web + mobile
`locacoes`	Cliente, veículo, datas, diária da época, valores, atraso, quem registrou e encerrou, **km de saída e retorno, assinatura, taxas extras e motivo**	Web + mobile
`manutencoes`	Defeito, oficina, datas, custo, **checklist (motor, freios, pneus, óleo)**	Web + mobile (laudo)
`logs`, `login_tentativas`	Histórico de ações e tentativas de login	Web

Ordem do trabalho

A regra continua: **primeiro conectar as três ao banco novo, depois completar as funcionalidades**. Se o desktop ficasse 100% pronto antes de a web e o celular entrarem, um erro no banco só apareceria perto da entrega. Conectando as três cedo, o teste cruzado ("alugar no celular, devolver no desktop, conferir na web") acontece no meio do caminho, quando ainda dá tempo de corrigir.

O que mudou com o prazo: depois que o banco sobe, **as três plataformas andam em paralelo**. Eu continuo fazendo um pacote por vez, mas, enquanto o Caio testa um pacote do desktop, eu já preparo o da web para o Gabriel testar. Cada um testa a própria plataforma, e ninguém fica parado esperando o outro.



As fases 2, 3 e 4 se sobrepõem de propósito: o banco é o mesmo, então cada um testa a sua plataforma ao mesmo tempo.

Calendário até 27/10

Quando	Fase	O que acontece	Quem testa
Ter 13 – Qua 14	0 · Preparo	Repositório organizado em pastas; GitHub Desktop instalado no notebook do Caio	Caio
Qua 14 – Qui 15	1 · Banco	Adapto o script de vocês e testo aqui; Caio roda no Supabase. **M1**	Caio
Qui 15 – Sáb 17	2 · Desktop conectar	8 pacotes. **M2** no sábado	Caio
Sex 16 – Seg 19	3 · Web conectar	8 pacotes. **M3**: teste cruzado desktop ↔ web	Gabriel
Dom 18 – Ter 20	4 · Mobile conectar	8 pacotes. **M4**: teste cruzado das três	Evaldo
Qua 21 – Sex 23	5 · Completar	Prioridade A primeiro, B se der tempo	Cada um na sua
Sáb 24	**Congelamento**	Daqui em diante, nada novo: só correção de erro	Todos
Sáb 24 – Dom 25	6 · Fechar	Dados de demonstração, roteiro do stand, README, versão 1.0	Todos
Seg 26	Ensaio geral	As três rodando no notebook do Caio, do jeito que vai ser no stand	Todos
Ter 27	**Entrega**		

> **O gargalo é o teste, não o código**
> Eu consigo entregar vários pacotes por dia; o que segura o ritmo é cada pacote ser testado e commitado. Se um pacote ficar um dia inteiro sem teste, o calendário escorrega. Sempre que possível, testem no mesmo dia em que o pacote chegar. O notebook do Caio roda as três plataformas, então ele também pode testar web e mobile quando o Gabriel ou o Evaldo não estiverem com o computador.

Pacotes (cada pacote = um commit)

Quem commita é uma sugestão para o histórico refletir o papel de cada um. Os `vocês` marcam o que precisa ser feito no painel do Supabase.

Fase 0 · Preparo (13–14/10)

Troca os zips do repositório pelas pastas de código, uma plataforma por commit. É só organização: o código não muda.

#	Pacote	Mensagem de commit	Quem commita
0.1	Estrutura de pastas, `.gitignore` (build, node_modules, senhas) e README geral	`chore: organiza o repositório em pastas por plataforma`	Caio
0.2	`desktop/` com o código da v3.3	`chore(desktop): adiciona o código do desktop v3.3`	Caio
0.3	`web/` com a versão completa de 9 de outubro	`chore(web): adiciona o código da web`	Gabriel
0.4	`mobile/` com o código da v13, sem a pasta `build`	`chore(mobile): adiciona o código do app v13`	Evaldo
0.5	Remove os zips antigos; os PDFs vão para `docs/`	`docs: documentação das plataformas e plano de unificação`	Caio
0.6	Permissão de internet no APK final do Android	`fix(mobile): libera internet no APK de release`	Evaldo

Fase 1 · Banco unificado (14–15/10)

Como vocês já têm o `royale_cars_supabase.sql`, a fase ficou menor: ele entra como está, e eu escrevo um complemento com o que o desktop e o celular trazem. Testo os dois juntos num PostgreSQL 16 aqui antes de entregar.

#	Pacote	Mensagem de commit	Quem commita
1.1	`banco/01_royale_cars_supabase.sql`: o script de vocês, sem mudança (base da web)	`feat(banco): script do banco unificado`	Gabriel
1.2	`banco/02_complemento.sql`: endereço, quilometragem, assinatura, taxas extras, checklist, modelos de grupo, bucket `contratos`, parâmetros novos nas funções e remoção das tabelas antigas (`usuarios`, `grupos_permissao`, `laudos_manutencao`)	`feat(banco): complemento com km, assinatura e checklist`	Evaldo
1.3	Edge Function `admin-usuarios` (se não vier junto com o script, eu escrevo)	`feat(supabase): função de administração de usuários`	Caio
1.4	`banco/03_dados_demo.sql` e roteiro de testes do banco	`test(banco): dados de demonstração e roteiro de testes`	Caio

M1 · banco no ar (Qui 15) vocês O Caio roda os scripts no SQL Editor, desliga o cadastro público, cria as contas de teste e publica a função. Os dados atuais são apagados (decisão 3). Eu mando o passo a passo com cada clique.

Fase 2 · Desktop conectado (15–17/10, commits do Caio)

#	Pacote	Mensagem de commit
2.1	Login pelo Supabase Auth, bloqueio e registro de tentativas; sai "Criar conta", "Esqueci minha senha" e `bcryptjs`	`feat(desktop): login pelo Supabase Auth`
2.2	Troca obrigatória de senha, saída por inatividade e Minha conta salvando de verdade	`feat(desktop): troca de senha obrigatória e saída automática`
2.3	Menu e processo principal pelas 13 chaves; grupos viram modelos	`refactor(desktop): permissões no padrão das 3 plataformas`
2.4	Colaboradores pela Edge Function (senha temporária, ativar, desativar)	`feat(desktop): administração de colaboradores pelo Supabase`
2.5	Clientes sem máscara no banco, validade da CNH, erros do banco na tela, e-mail e endereço salvando na edição	`refactor(desktop): clientes no padrão unificado`
2.6	Frota: sem status no formulário, placa sem hífen, ano, chassi e km	`refactor(desktop): frota no padrão unificado`
2.7	Locações pelas funções: retirada com km; devolução com data real, atraso, km, taxas e defeito; cancelar	`feat(desktop): locações pelas funções do banco`
2.8	Dashboard e Financeiro lendo as visões (corrige o ticket médio)	`refactor(desktop): painéis lendo as visões do banco`

M2 · primeiro app no banco novo (Sáb 17). O Caio faz o roteiro de testes no desktop; o que der errado no banco é corrigido antes de a web e o celular avançarem.

Fase 3 · Web conectada (16–19/10, commits do Gabriel)

#	Pacote	Mensagem de commit
3.1	`includes/supabase.php` (login, consultas, rpc, upload) e configuração fora do Git	`feat(web): cliente do Supabase em PHP`
3.2	Login, sessão, bloqueio e troca de senha pelo Supabase Auth	`feat(web): login pelo Supabase Auth`
3.3	Clientes, ficha e arquivados (+ endereço)	`refactor(web): clientes no Supabase`
3.4	Frota, foto no Storage e quilometragem	`refactor(web): frota e fotos no Supabase`

#	Pacote	Mensagem de commit
3.5	Locações e manutenção pelas funções (+ km, taxas, checklist, assinatura no comprovante)	`feat(web): locações e manutenção pelas funções do banco`
3.6	Dashboard, alertas, financeiro e relatórios pelas visões	`refactor(web): painéis e relatórios pelas visões`
3.7	Administração pela Edge Function; histórico e tentativas	`feat(web): administração pelo Supabase`
3.8	Remove a conexão MySQL; `royale_cars.sql` vai para `docs/legado/`	`chore(web): remove MySQL`

M3 · teste cruzado desktop ↔ web (Seg 19). Cadastrar cliente na web e alugar no desktop; desativar alguém na web e ver o desktop recusar.

Fase 4 · Mobile conectado, foco no atendimento (18–20/10, commits do Evaldo)

Pela decisão 9, o celular fica com clientes, frota, retirada com assinatura, devolução, manutenção e alertas. Financeiro e relatórios só para ler; a administração da equipe fica na web e no desktop.

#	Pacote	Mensagem de commit
4.1	Login pelo Supabase Auth, sessão salva no aparelho, troca obrigatória; sai "Criar conta" e `bcrypt`	`feat(mobile): login pelo Supabase Auth`
4.2	Permissões nos atalhos e botões; tira o `id_empresa` das consultas	`feat(mobile): menu por permissão`
4.3	Clientes no padrão + tela de arquivados	`refactor(mobile): clientes no padrão unificado`
4.4	Frota no padrão + arquivados	`refactor(mobile): frota no padrão unificado`
4.5	Nova locação pela função; assinatura vai para o Storage	`feat(mobile): retirada pela função com assinatura no Storage`
4.6	Lista de locações com abas, devolução pela função, cancelar e contrato PDF a qualquer momento	`feat(mobile): devolução, cancelamento e contrato`
4.7	Manutenção: checklist + oficina + botão Liberar com custo	`feat(mobile): manutenção completa`
4.8	Painel com cartões e alertas; Financeiro e Relatórios só leitura pelas visões; Minha conta	`feat(mobile): painel, alertas e minha conta`

M4 · teste cruzado das três (Ter 20). Alugar no celular (com assinatura), devolver no desktop, conferir o comprovante e o financeiro na web.

Fase 5 · Completar (21–23/10), por prioridade

Com 14 dias, não dá para fazer tudo nas três. A ordem abaixo põe primeiro o que a banca vai ver no stand.

Prioridade	#	Pacote	Quem commita
A · essencial	5.1	Desktop: tela de Manutenção (com checklist e liberar)	Caio
A · essencial	5.2	Desktop: contrato/comprovante com assinatura para imprimir	Caio
A · essencial	5.3	Desktop: alertas e dashboard completo (gráfico e barra da frota)	Caio
A · essencial	5.4–5.6	Nomes de tela, textos e Sobre Nós iguais (um commit por plataforma)	Cada um na sua
B · se der	5.7	Desktop: fichas de cliente e veículo	Caio
B · se der	5.8	Foto do veículo no desktop e no celular	Caio / Evaldo
B · se der	5.9	Desktop: relatórios simples	Caio
C · só na web	—	Relatórios com CSV, histórico de ações e tentativas de login. No stand, mostrar na web: "o painel completo do chefe fica no computador da gerência".	

Fase 6 · Fechamento (24–26/10)

#	Pacote	Quem commita
6.1	Dados de demonstração finais e roteiro do stand nas três plataformas	Gabriel
6.2	README e documentação final em `docs/`	Evaldo
6.3	Versão 1.0 marcada no GitHub (tag e release)	Caio

Como fazer os commits sem saber Git

Hoje vocês usam o site do GitHub ("Add files via upload") nos computadores do Cotil. Isso funciona para mandar arquivos novos, mas tem dois problemas para o nosso caso: o site não apaga arquivos em lote (cada arquivo apagado é um clique e um commit) e aceita no máximo 100 arquivos por envio. Por isso proponho dois jeitos; vocês escolhem.

Jeito A (recomendado): GitHub Desktop no notebook do Caio

O **GitHub Desktop** é um programa gratuito, com botões, que já instala o Git junto. Ninguém precisa decorar comando. Ele mostra a lista de arquivos que mudaram e, ao clicar em um, mostra as linhas novas em verde e as apagadas em vermelho. Isso também ajuda a estudar o código de cada pacote.

1. Instalar em `desktop.github.com` e entrar com a conta do GitHub do Caio.
2. **File → Clone repository** → escolher `projeto_royale_cars` → escolher uma pasta no computador.

3. No app do Claude, adicionar essa pasta a esta conversa (botão "Adicionar pasta"). A partir daí **eu escrevo os arquivos direto nela**, sem zip, sempre depois de vocês aprovarem.
4. A cada pacote: testar, abrir o GitHub Desktop, conferir a lista de mudanças, colar a mensagem de commit, clicar em **Commit to main** e depois em **Push origin**.

> **Os commits de cada um continuam aparecendo no perfil de cada um**
> Na caixa de commit do GitHub Desktop há um botão de **co-autores** (ícone de pessoa com +). Num pacote da web, o Caio adiciona o Gabriel como co-autor; num do mobile, o Evaldo. O GitHub mostra os dois nomes no commit e conta a contribuição nos dois perfis. É o jeito honesto de registrar trabalho feito em dupla no mesmo computador.

Jeito B: cada um pelo site, no Cotil

1. Eu mando um zip só com os arquivos que mudaram, já nas pastas certas, e a lista dos arquivos que devem ser apagados.
2. No GitHub: **Add file → Upload files**, arrastar as pastas do zip, colar a mensagem de commit e clicar em **Commit changes**.
3. Para cada arquivo da lista de apagar: abrir o arquivo no site → ícone de lixeira → commit.

Funciona sem instalar nada, mas é mais lento, e quem testa precisa ter o projeto rodando no computador do Cotil.

Dá para misturar os dois: o dia a dia pelo jeito A, e o Gabriel e o Evaldo commitando pelo site os pacotes que testarem no Cotil.

Estrutura do repositório depois da Fase 0

```
projeto_royale_cars/
├── banco/                  scripts SQL numerados, na ordem em que rodam
├── supabase/functions/     admin-usuarios (Edge Function)
├── desktop/                Electron
├── web/                    PHP
├── mobile/                 Flutter
├── docs/                   os PDFs das plataformas e este plano
├── .gitignore              build/, node_modules/, config com senha
└── README.md               o que é o projeto e como rodar cada parte
```

Padrão das mensagens de commit

Formato `tipo(área): o que mudou`, em português e no imperativo, como nos pacotes acima.

Tipo	Quando usar	Exemplo
`feat`	Funcionalidade nova	`feat(mobile): manutenção completa`
`fix`	Correção de erro	`fix(desktop): salva e-mail ao editar cliente`
`refactor`	Muda o código sem mudar o que a pessoa vê	`refactor(web): clientes no Supabase`

Tipo	Quando usar	Exemplo
`chore`	Organização, configuração	`chore: organiza o repositório em pastas`
`docs` / `test`	Documentação / testes	`docs: roteiro do stand`

O que vem em cada entrega

Parte	O que é
Os arquivos	Escritos direto na pasta (jeito A) ou num zip (jeito B).
O que foi feito	Lista curta dos arquivos que mudaram e o que mudou em cada um.
Como testar	Passo a passo do teste ("entre como funcionário, tente abrir Financeiro, deve aparecer Acesso negado").
Mensagem de commit	Pronta para copiar, com o co-autor sugerido.
Precisa de vocês?	Se tiver algo para rodar no Supabase, vem aqui, com os cliques.
Aula do pacote	O raciocínio do código: o problema, a ideia e como ela vira linha de código. É o que vocês vão precisar para responder a banca.

Aprovação: no começo de cada fase eu mostro a lista de pacotes; com o "pode fazer", entrego em sequência e só paro para perguntar quando aparecer algo fora do previsto.

Cuidados

- **Nunca commitar a chave** `service_role` do Supabase nem senhas. A chave `publishable` pode ir; a de serviço só existe no painel do Supabase.
- A configuração da web com dados de conexão fica num arquivo fora do Git (`config.local.php`), com um modelo `config.exemplo.php` no repositório.
- Confiram as regras do curso sobre uso de IA. Se for preciso declarar, uma linha no README ("desenvolvido com apoio do Claude, da Anthropic") resolve.
- Quem commita precisa entender o pacote: a banca vai perguntar sobre o código. Por isso cada entrega vem com a aula.

Riscos e como evitamos

Risco	Como evitamos
Prazo de 14 dias	Plataformas em paralelo depois do banco; prioridades A, B e C na Fase 5; congelamento no dia 24.
Pacote parado sem teste	Testar no mesmo dia; o notebook do Caio roda as três e serve de reserva.
Erro no banco aparecer tarde	Script testado aqui num PostgreSQL 16, roteiro de testes no pacote 1.4 e teste cruzado a cada plataforma que entra.

Risco	Como evitamos
Código Flutter que eu não consigo compilar	Pacotes pequenos; quem testa roda `flutter run` (no celular ou no Chrome) e manda o erro; eu corrijo num pacote `fix`.
Edge Function ser uma ferramenta nova	Publicação pelo próprio painel do Supabase, com passo a passo. Plano B: uma função SQL que só o chefe chama.
Algo quebrar perto da entrega	Tag de versão a cada marco (M1 a M4): se der errado, dá para voltar ao último ponto que funcionava.

Decisões do grupo (respondidas em 13/10)

#	Decisão	O que ficou decidido
1	Uma empresa só ou várias	**Uma só.** O `id_empresa` sai do desktop e do celular.
2	Login	**Supabase Auth nas três**, só por e-mail.
3	Dados atuais do Supabase	**Podem ser apagados** (5 usuários, 8 clientes, 14 veículos, 20 locações, 13 grupos, 2 laudos). Entram dados de demonstração.
4	Grupos de permissão do desktop	**Viram modelos** que marcam as caixas da pessoa.
5	Quilometragem	**Nas três.**
6	Assinatura	**Arquivo no Storage**, colhida no celular; web e desktop mostram. Obrigatória só no celular.
7	Taxas na devolução	**Taxas extras com motivo**, separadas das diárias e do atraso.
8	Endereço do cliente	**Nas três**, opcional.
9	Telas do celular	**Foco no atendimento.** Financeiro e relatórios só leitura; administração na web e no desktop.
10	Devolução adiantada	**Cobra só os dias usados.**
11	Troca do e-mail de login	**Só o chefe.**
12	Nomes de tela	**Tabela da seção Padrão comum.**
13	Sobre Nós e Contato	**Sobre Nós fica.** O formulário de Contato que finge enviar sai.

O que eu preciso de vocês agora

1. **A pasta `supabase/` inteira** (o `royale_cars_supabase.sql` e, se existir, a pasta `functions/admin-usuarios`). Pode mandar zipada.
2. **O jeito de commitar**: A (GitHub Desktop no notebook do Caio), B (pelo site, no Cotil) ou os dois.
3. **Se for o jeito A**: instalar o GitHub Desktop, clonar o `projeto_royale_cars` e adicionar a pasta clonada nesta conversa.

4. **O "pode fazer" para a Fase 0** (pacotes 0.1 a 0.6).

Acompanhamento:

- [ ] Pasta supabase/ enviada
- [ ] Jeito de commitar escolhido
- [ ] Fase 0 · repositório em pastas
- [ ] M1 · banco unificado no ar (15/10)
- [ ] M2 · desktop no banco novo (17/10)
- [ ] M3 · teste cruzado desktop ↔ web (19/10)
- [ ] M4 · teste cruzado das três (20/10)
- [ ] Prioridade A da Fase 5 (23/10)
- [ ] Congelamento (24/10)
- [ ] Ensaio geral (26/10)
- [ ] Entrega (27/10)